ISyE/Math/CS 425
Introduction to Combinatorial Optimization

1. Problems and Algorithms

Prof. Carla Michini
University of Wisconsin-Madison

Outline

- We will see two Combinatorial Optimization problems:
  - The Traveling Salesman Problem.
  - The Matching Problem.
- We will see their similarities and differences.
- We will understand the size of a problem and the running time of an algorithm.

1.1 Two Problems

Oil company problem

- Oil company.
- 47 drilling platforms.
- Each platform has a set of controls to regulate rate of flow.
- Periodically it is necessary to visit certain platforms to regulate rates of flows.
- Traveling is done by helicopter, which leaves from and returns to the helicopter base.
- Oil company wants a method for routing the helicopter in such a way that the required platforms are visited, and the total flying time is minimized.
- We assume that the flying time is proportional to the distance traveled.

Euclidean Traveling Salesman Problem (TSP)

Euclidean TSP

- Input: a set V of points in the Euclidean plane: $V = \{(x_i, y_i) : i = 1, \ldots, n\}$.
- Distance between (x_i, y_i) and (x_j, y_j) is

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

- Objective: find a tour passing through each point of V, for which the sum of the distances is minimized.

- V consists of the platforms to be visited, plus the helicopter base.
- Is it profitable to pass through a platform more than once?

Euclidean Traveling Salesman Problem (TSP)

Euclidean TSP

- Input: a set V of points in the Euclidean plane: $V = \{(x_i, y_i) : i = 1, \ldots, n\}$.
- Distance between (x_i, y_i) and (x_j, y_j) is

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

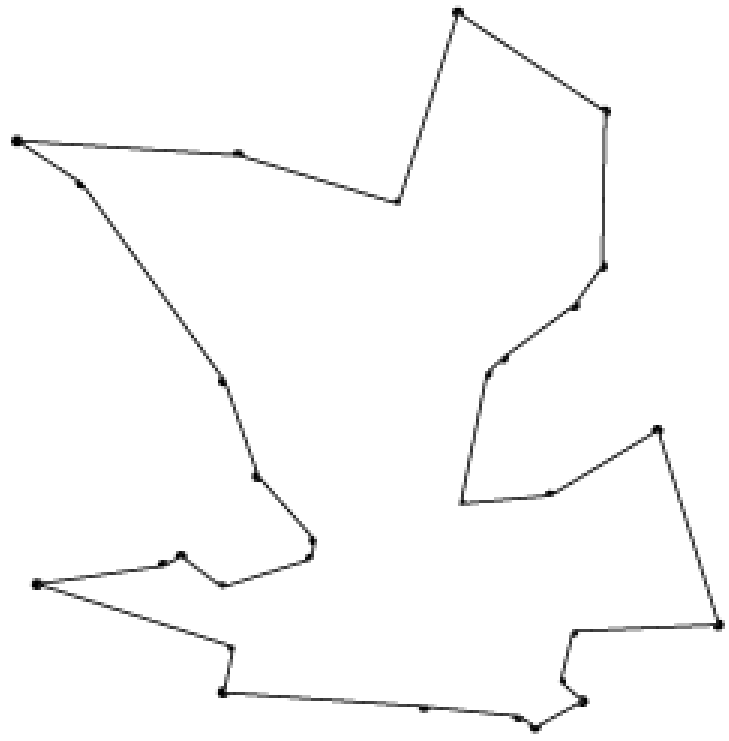
- Objective: find a tour passing through each point of V once, for which the sum of the distances is minimized.

- V consists of the platforms to be visited, plus the helicopter base.
- Is it profitable to pass through a platform more than once?

Example



Instance



Optimal solution

Brute force algorithm

Brute force algorithm. Finite number of solutions!

1. List all ..... different possible solutions.
2. Evaluate them.
3. Pick the best.

If it takes one nanosecond ($= 10^{-9}$ seconds) to evaluate a single solution, then for $n = 23$, it takes 178 centuries for the algorithm to terminate!

Brute force algorithm

Brute force algorithm. Finite number of solutions!

1. List all $(n-1)!/2$ different possible solutions.
2. Evaluate them.
3. Pick the best.

If it takes one nanosecond ($= 10^{-9}$ seconds) to evaluate a single solution, then for $n = 23$, it takes 178 centuries for the algorithm to terminate!

Brute force algorithm

Brute force algorithm. Finite number of solutions!

1. List all $(n-1)!/2$ different possible solutions.
2. Evaluate them.
3. Pick the best.

If it takes one nanosecond ($= 10^{-9}$ seconds) to evaluate a single solution, then for $n = 23$, it takes 178 centuries for the algorithm to terminate!

Question: How does 2^n compare with $n!$?

Nearest Neighbor Algorithm

Nearest Neighbor Algorithm. Faster method, but does not guarantee to produce the optimal solution.

1. Pick any starting point.
2. Go to the nearest point not yet visited.
3. Repeat this until all points have been visited.
4. Return to the starting point.

Nearest Neighbor Algorithm

Nearest Neighbor Algorithm. Faster method, but does not guarantee to produce the optimal solution.

1. Pick any starting point.
2. Go to the nearest point not yet visited.
3. Repeat this until all points have been visited.
4. Return to the starting point.

Video example

Nearest Neighbor Algorithm

Nearest Neighbor Algorithm. Faster method, but does not guarantee to produce the optimal solution.

1. Pick any starting point.
2. Go to the nearest point not yet visited.
3. Repeat this until all points have been visited.
4. Return to the starting point.

Video example

Question: Why the final tour in the video is not optimal?

Nearest Neighbor Algorithm

Nearest Neighbor Algorithm. Faster method, but does not guarantee to produce the optimal solution.

1. Pick any starting point.
2. Go to the nearest point not yet visited.
3. Repeat this until all points have been visited.
4. Return to the starting point.

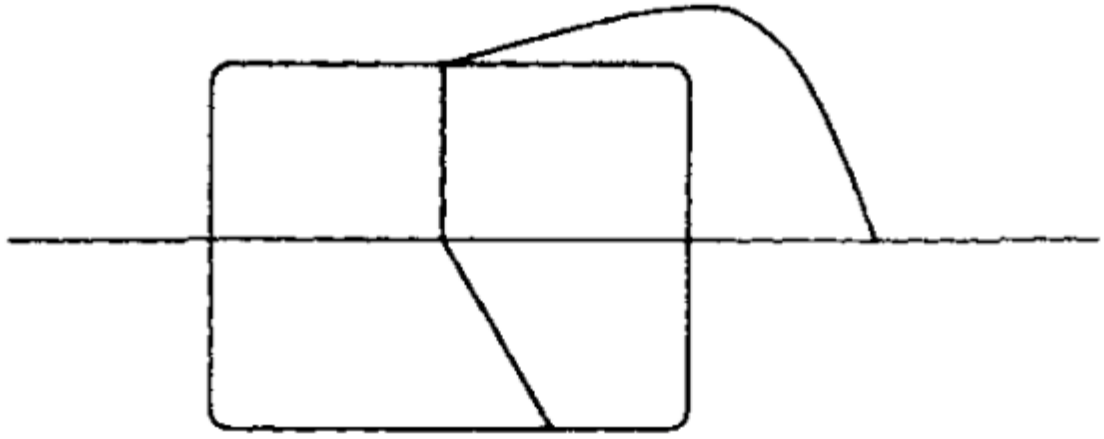
Video example

Question: Why the final tour in the video is not optimal?
Question: Can you construct a class of instances of the Euclidian TSP where the Nearest Neighbor Algorithm performs "badly"?

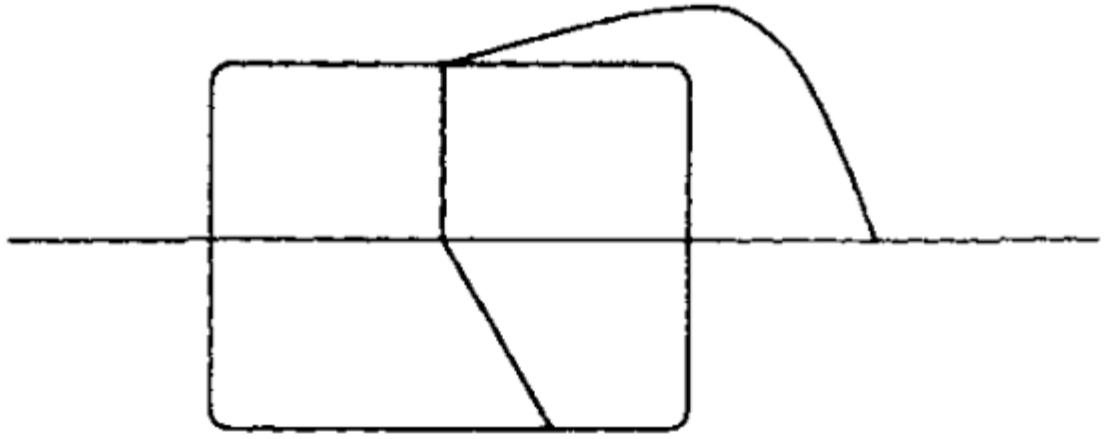
Circuit drawing problem

- Uses a plotter to draw a logic circuit
- Plotter operates by moving a pen back and forth, and simultaneously rolling a sheet of paper forward and backwards
- At the end, the pen should return to the starting position
- The problem is to minimize the time required to draw the figure:
  - "pen-down" time, when actually drawing +
  - "pen-up" time, when the pen is not contacting the paper,
- Question: Is it profitable, during "pen-down" time, to draw a line twice?

Circuit drawing problem



Circuit drawing problem

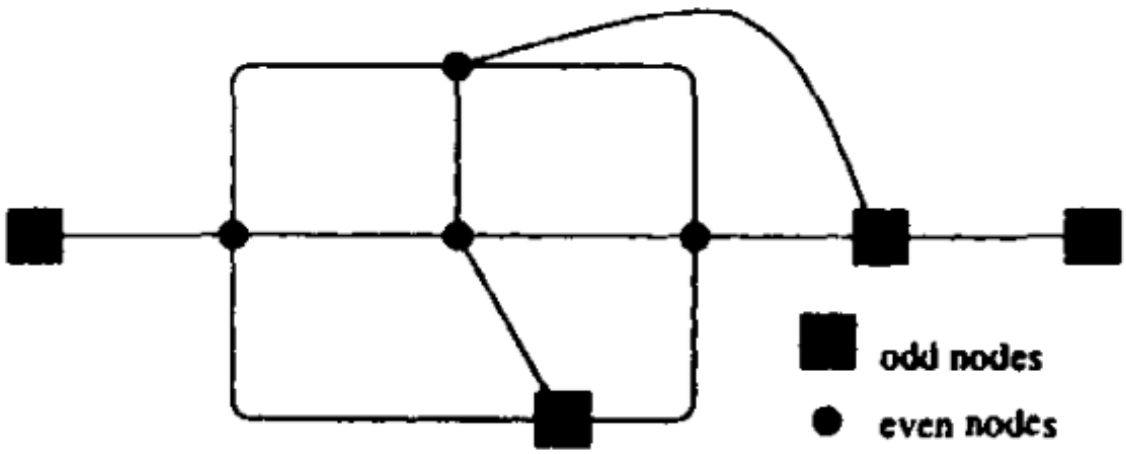


- Pen-down time is fixed, but we can reduce the pen-up time.
- Some amount of pen-up motion is necessary.

Question: Why?

Circuit drawing problem

- A node is a point of the figure from which a positive number of lines, other than 2, emanates.
- A node is odd if there is an odd number of lines coming out, and even otherwise.



Circuit drawing problem

Theorem

The figure can be drawn, returning to the starting point, without tracing the same line twice and with no pen-up motion *if and only if* it is connected and there are no odd nodes.

Circuit drawing problem

Theorem
The figure can be drawn, returning to the starting point, without tracing the same line twice and with no pen-up motion *if and only if* it is connected and there are no odd nodes.

Theorem
There is an even number of odd nodes.

Why?

Circuit drawing problem

Theorem
The figure can be drawn, returning to the starting point, without tracing the same line twice and with no pen-up motion *if and only if* it is connected and there are no odd nodes.

Theorem
There is an even number of odd nodes.

Why?

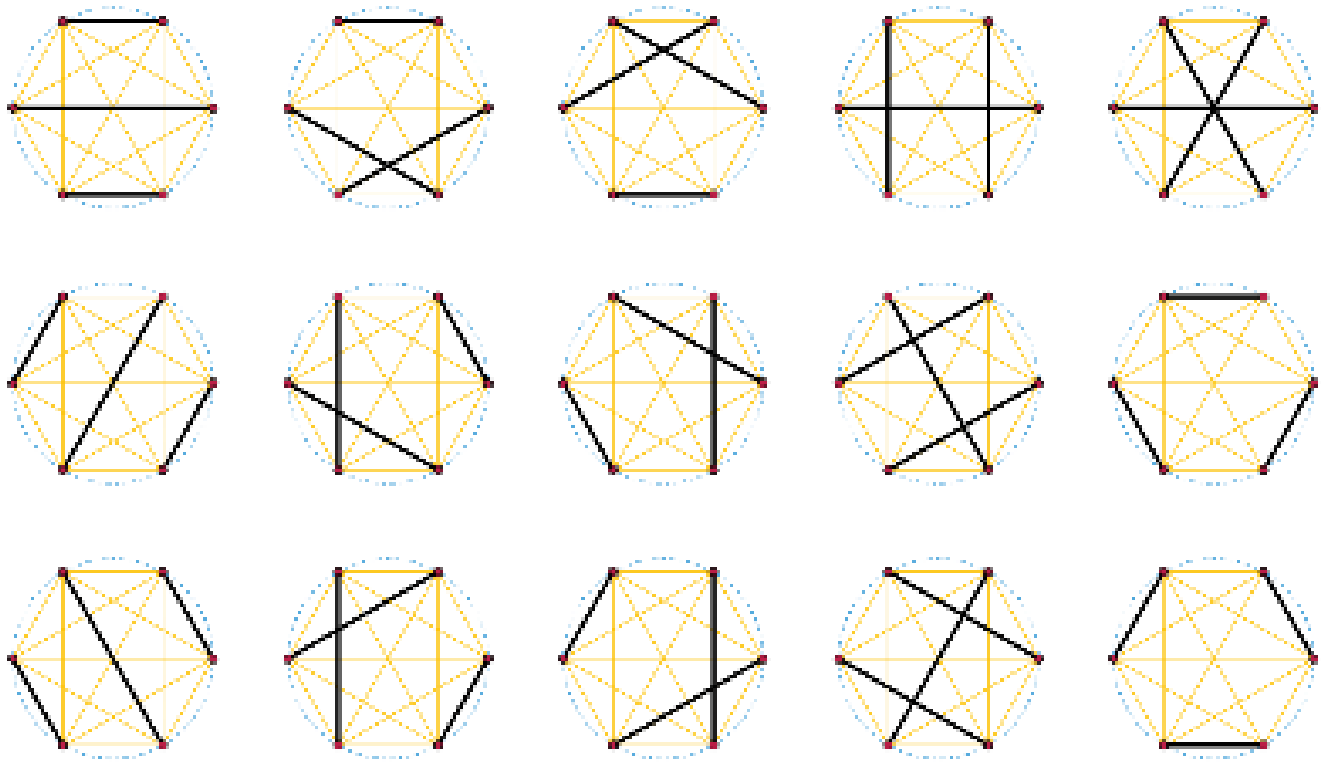
- We minimize pen-up motion by finding a set of new lines that we can add to the figure turning every odd node into an even node, and such that the total traversal time of the new lines is as small as possible.
- We assume that the time required to draw a line is proportional to the distance.

Euclidean matching problem

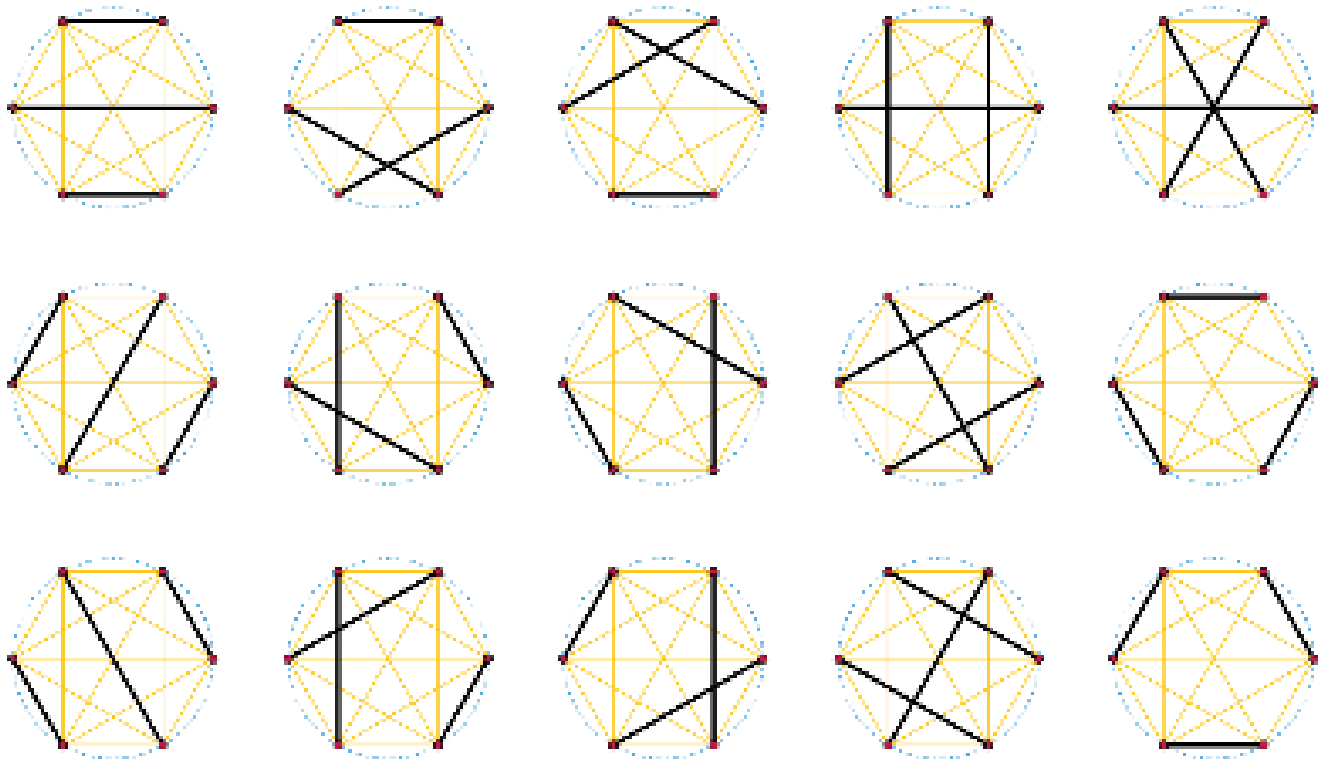
Euclidean matching problem

- Input: a set V of points in the Euclidean plane, $|V|$ even.
- Objective: find a set of lines, such that each point is an end of exactly one line, and such that the sum of the lengths of the lines is minimized.

Example



Example



Question: Which one is optimal?

Similarities and differences

Similarities:

- Selecting sets of lines connecting points in the plane.
- The number of feasible solutions is far too large to consider them all in a reasonable amount of time.
- Most simple heuristics for the problems do not perform very well.

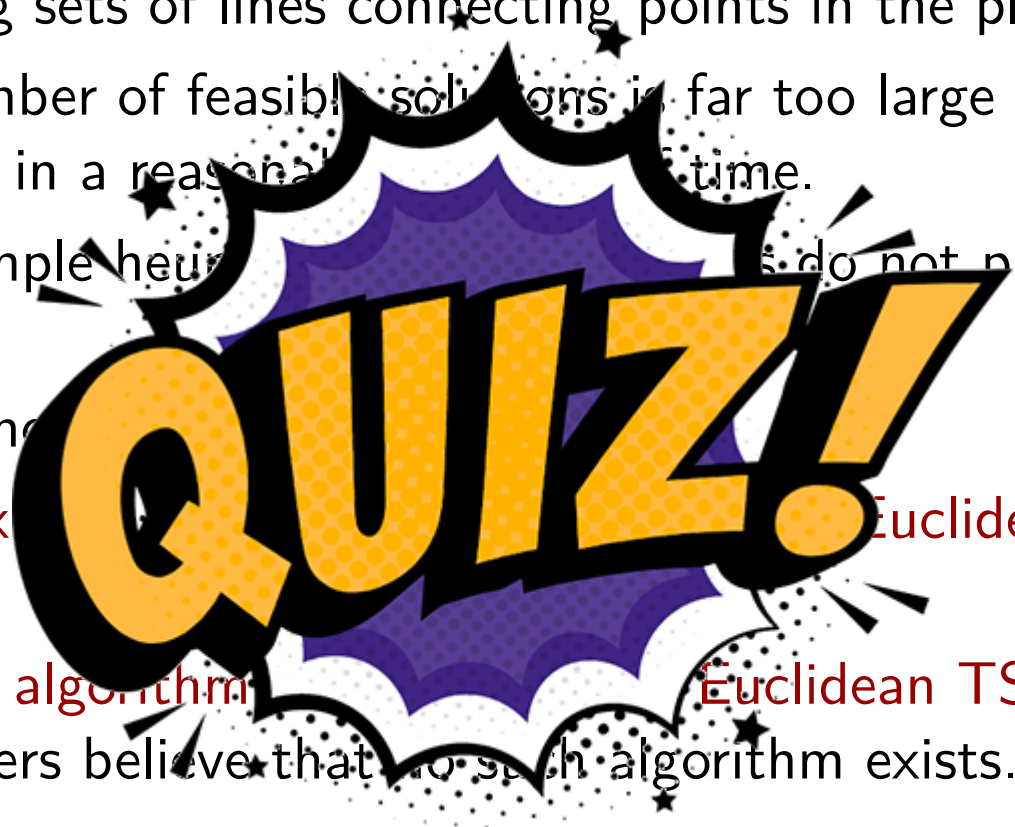
Major difference:

- There exists an efficient algorithm for the Euclidean matching problem.
- No such algorithm is known for the Euclidean TSP, and most researchers believe that no such algorithm exists.
- If there would exist an efficient algorithm for Euclidean TSP, then there would also exist an efficient algorithm for about every combinatorial optimization problem.

Similarities and differences

Similarities:

- Selecting sets of lines connecting points in the plane.
- The number of feasible solutions is far too large to consider them all in a reasonable amount of time.
- Most simple heuristics do not perform very well.



Major differences:

- There exists a polynomial-time algorithm for the Euclidean matching problem.
- No such algorithm is known for Euclidean TSP, and most researchers believe that no such algorithm exists.
- If there would exist an efficient algorithm for Euclidean TSP, then there would also exist an efficient algorithm for about every combinatorial optimization problem.

1.2 Measuring Running Times

Measuring running times

- What is an "efficient" algorithm?
- We estimate the efficiency by giving an upper bound on the number of steps it requires to solve a problem of a given size.
- This only provides an estimate of an algorithm's efficiency (because the bounds are taken over all possible instances of a given problem).
- Having a better bound does not necessarily mean better performance in practice.
- We count the number of "elementary" operations in the execution of the algorithm: read, write, addition, multiplication, division, comparison.

Size of a problem

- A combinatorial optimization problem consists of a discrete structure, together with a set of numbers.
- The size of such a problem is the length of an encoding of the structure plus the cardinality of the set of numbers used.
- Given an instance of a problem, let s be its size.
  We can say "the number of steps is bounded by $5s^2 + 3s$".

Size of a problem

- A combinatorial optimization problem consists of a discrete structure, together with a set of numbers.
- The size of such a problem is the length of an encoding of the structure plus the cardinality of the set of numbers used.
- Given an instance of a problem, let s be its size.
  We can say "the number of steps is bounded by $5s^2 + 3s$".

Example:

- An instance of the Euclidean TSP can be specified by giving the (x_i, y_i) coordinates of the n points to be visited.
- So the size of an instance is $s = 2n$.

Magnitude of the bound

- We are interested in the performance on instances of large size.
- We concentrate on the magnitude of the bound: $5s^2 + 3s$ is "order s^2".
- Formally: if $f(s)$ and $g(s)$ are positive real-valued functions on the set of nonnegative integers, we say $f(s) \in O(g(s))$ if there exists a constant $c > 0$ such that $f(s) \leq c \cdot g(s)$ for all large enough values of s.
  Examples: $5s^2 + 3s \in O(s^2)$ and $35 \cdot 2^s + s^3 \in O(2^s)$.
- Efficient algorithms are the ones that have running-time bounds of $O(s^k)$, for some $k \in \mathbb{N}$.
- These algorithms are called polynomial-time algorithms, and their running times does not increase too rapidly as the problem size increases.

Example: Nearest Neighbor Algorithm for TSP

Nearest Neighbor Algorithm

1. Pick any starting point.
2. Go to the nearest point not yet visited.
3. Repeat this until all points have been visited.
4. Return to the starting point.

Example: Nearest Neighbor Algorithm for TSP

- Initialization: initialize one object with three fields per point $(x_i, y_i, mark = 0)$. $\boxed{3n \text{ steps}}$
- Pass $\boxed{n-1 \text{ times}}$: take a point v_j, scan through all other objects (at most $\boxed{n-1 \text{ times}}$). For each object:
  - Check if the mark is equal to 0.
    (1 compare = $\boxed{1 \text{ step}}$)
  - Compute the distance^2 to v_j and store it.
    (3 additions + 2 multiplications + 1 write = $\boxed{6 \text{ steps}}$)
  - Keep track of the point v_{i^*} that has the least distance^2.
    (1 comparison + 1 write = $\boxed{2 \text{ steps}}$)

  Then output v_{i^*} as the next point in the tour $\boxed{1 \text{ step}}$, set v_{i^*}'s mark field to 1 $\boxed{1 \text{ step}}$ and continue the search from v_{i^*}.

At most $3n + (n-1)((n-1) \cdot 9 + 2)$ steps $\in O(n^2)$.

Example: Nearest Neighbor Algorithm for TSP

- The size of an instance of the Euclidean TSP is $s = 2n$.
- Number of steps $\in O(n^2) = O((\frac{s}{2})^2)$, thus $\in O(s^2)$.
- Nearest Neighbor Algorithm is a polynomial-time algorithm.